Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

†Work performed while at Google Brain.

‡Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

注意力机制就是一切

阿什什·瓦斯瓦尼 谷歌大脑 avaswani@google.com

诺姆·沙泽尔 谷歌大脑 noam@google.com

尼基·帕尔玛 谷歌研究院 nikip@google.com

雅各布·乌兹科雷特 谷歌研究院 usz@google.com

利昂·琼斯 谷歌研究院 llion@google.com

艾丹·N·戈麦斯 多伦多大学 aidan@cs.toronto.edu

武卡什·凯撒 谷歌大脑 lukaszkaiser@google.com

伊利亚·波洛苏欣 illia.polosukhin@gmail.com

摘要

主流的序列转换模型依赖于复杂的循环或卷积神经网络，这些网络包含编码器和解码器。性能最佳的模型还通过注意力机制连接编码器和解码器。我们提出了一种新的简单网络架构——Transformer，它完全基于注意力机制，彻底摒弃了循环和卷积结构。在两个机器翻译任务上的实验表明，该模型在质量上更优，同时具有更高的并行化能力且训练时间显著缩短。我们的模型在 WMT 2014 英语到德语翻译任务中取得了 28.4 的 BLEU 分数，比现有最佳结果（包括集成模型）提高了超过 2 个 BLEU。在 WMT 2014 英语到法语翻译任务中，我们的模型仅用 8 块 GPU 训练 3.5 天，就以 41.8 的 BLEU 分数创造了新的单模型最优记录，其训练成本仅为文献中最佳模型的一小部分。通过成功应用于英语成分句法分析（无论训练数据量大小），我们证明了 Transformer 能良好泛化至其他任务。

同等贡献，作者排序随机。Jakob 提出用自注意力机制替代 RNN 并率先评估该设想；Ashish 与 Illia 共同设计并实现了首个 Transformer 模型，全程深度参与各项工作；Noam 提出了缩放点积注意力、多头注意力及无参数位置表征方案，几乎参与了所有细节的讨论；Niki 在我们原始代码库和 tensor2tensor 中设计、实现、调优并评估了无数模型变体；Llion 尝试了多种新颖模型架构，负责初始代码库构建、高效推理及可视化实现；Lukasz 和 Aidan 耗费大量时间设计 tensor2tensor 各模块并完成实现，取代旧代码库显著提升结果并极大加速研究进程。

†工作完成于谷歌大脑任职期间。

‡工作完成于谷歌研究院任职期间。

第 31 届神经信息处理系统大会（NIPS 2017），美国加州长滩市。

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t, as a function of the previous hidden state h_{t-1} and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

2 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [16], ByteNet [18] and ConvS2S [9], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [12]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2.

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 27, 28, 22].

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [34].

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [17, 18] and [9].

3 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 35]. Here, the encoder maps an input sequence of symbol representations $(x_1, ..., x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, ..., z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, ..., y_m)$ of symbols one element at a time. At each step the model is auto-regressive [10], consuming the previously generated symbols as additional input when generating the next.

1 引言

循环神经网络，尤其是长短期记忆网络[13]和门控循环神经网络[7]，已被牢固确立为序列建模和转换问题（如语言建模和机器翻译[35, 2, 5]）中的最先进方法。此后，众多研究持续推动着循环语言模型与编码器-解码器架构的发展边界[38, 24, 15]。

循环模型通常沿着输入和输出序列的符号位置分解计算过程。通过将位置与计算时间步骤对齐，它们生成一系列隐藏状态 h，作为前一个隐藏状态 h 和位置 t 输入的函数。这种固有的顺序特性阻碍了训练样本内的并行化，这在序列长度较长时变得尤为关键，因为内存限制制约了跨样本的批处理。近期研究通过分解技巧[21]和条件计算[32]显著提升了计算效率，后者还同时改善了模型性能。然而，顺序计算的根本限制依然存在。

注意力机制已成为各类任务中构建引人注目的序列建模与转换模型不可或缺的部分，它能够无视输入或输出序列中的距离来建模依赖关系[2,19]。然而，除少数情况外[27]，此类注意力机制通常与循环网络结合使用。

在本研究中，我们提出了 Transformer 模型架构，它摒弃了循环结构，完全依赖注意力机制来捕捉输入与输出间的全局依赖关系。该架构显著提升了并行化能力，仅需在八块 P100 GPU 上训练十二小时，即可达到翻译质量的新最优水平。

2 背景

减少序列计算的目标同样构成了扩展神经 GPU[16]、ByteNet[18]和 ConvS2S[9]的基础，这些模型均采用卷积神经网络作为基本构建模块，并行计算所有输入与输出位置的隐藏表示。在这些模型中，关联两个任意输入或输出位置信号所需的操作次数随位置间距增长而增加：ConvS2S 呈线性增长，ByteNet 呈对数增长。这使得学习远距离位置间的依赖关系更为困难[12]。在 Transformer 中，这一操作次数被降至常数级别，尽管代价是由于对注意力加权位置进行平均化而导致有效分辨率降低——我们通过 3.2 节所述的多头注意力机制来抵消这一效应。

自注意力机制，有时也称为内部注意力，是一种通过关联单一序列中不同位置以计算该序列表示的注意力机制。自注意力已在多项任务中成功应用，包括阅读理解、抽象摘要、文本蕴含及学习与任务无关的句子表示[4,27,28,22]。

端到端记忆网络基于循环注意力机制而非序列对齐的递归，并已在简单语言问答和语言建模任务中展现出优异性能[34]。

然而，据我们所知，Transformer 是首个完全依赖自注意力机制计算输入输出表示，而无需使用序列对齐的 RNN 或卷积的转换模型。在接下来的章节中，我们将描述 Transformer 架构，阐释自注意力的动机，并讨论其相较于[17,18]和[9]等模型的优势。

3 模型架构

大多数具有竞争力的神经序列转换模型采用编码器-解码器结构[5,2,35]。其中，编码器将输入符号表示序列(x,...,x)映射为连续表示序列 z=(z,...,z)。给定 z 后，解码器逐步生成输出符号序列(y,...,y)，每次生成一个元素。该模型在每一步都是自回归的[10]，生成下一个符号时会将先前生成的符号作为额外输入。

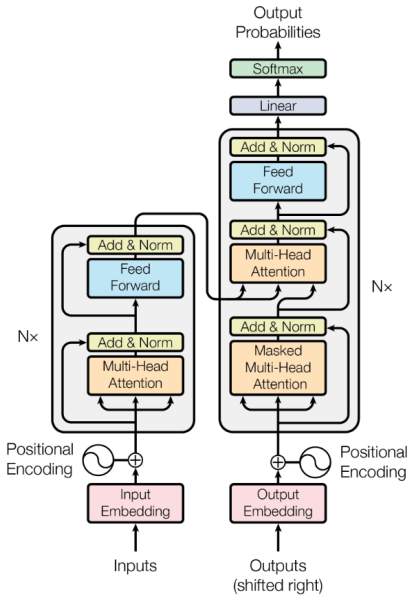


Figure 1: The Transformer - model architecture.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$, where $\mathrm{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum

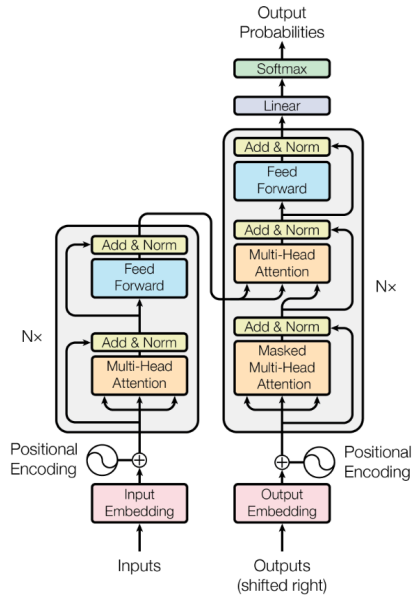


图 1：Transformer 模型架构。

Transformer 遵循这一整体架构，编码器和解码器分别采用堆叠的自注意力机制和逐点全连接层（如图 1 左右两部分所示）。

3.1 编码器与解码器堆栈

编码器：该编码器由 N=6 个相同层堆叠而成。每层包含两个子层。第一子层是多头自注意力机制，第二子层则是简单的位置全连接前馈网络。我们在每个子层周围采用了残差连接[11]，随后进行层归一化[1]。即，每个子层的输出为 LayerNorm(x + Sublayer(x))，其中 Sublayer(x)表示该子层自身实现的函数。为便于这些残差连接，模型中所有子层及嵌入层的输出维度均设为 d=512。

解码器：解码器同样由 N=6 个相同层堆叠而成。除编码器每层中的两个子层外，解码器还插入了第三个子层，该子层对编码器堆栈的输出执行多头注意力机制。与编码器类似，我们在每个子层周围采用残差连接，并随后进行层归一化。我们还调整了解码器堆栈中的自注意力子层，以防止位置关注到后续位置。这种掩蔽机制，加上输出嵌入向右偏移一个位置的设计，确保了位置 i 的预测仅能依赖于小于 i 的已知输出。

3.2 注意力

注意力函数可描述为将查询与一组键值对映射至输出的过程，其中查询、键、值及输出均为向量。输出通过加权求和的方式计算得出。

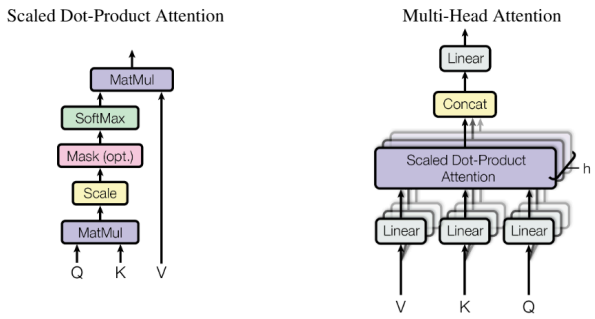


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

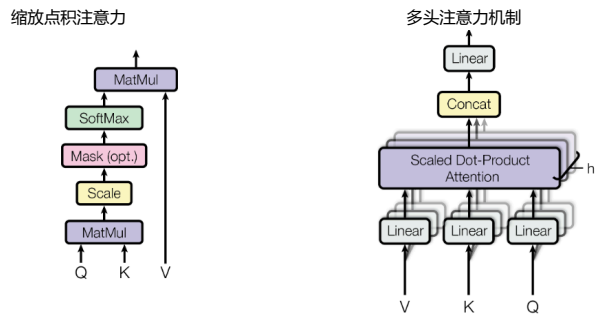
The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients [4]. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model}-dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional

[4] To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k.



输出计算为加权和。图 2：（左）缩放点积注意力机制。（右）多头注意力由多个并行运行的注意力层组成。

其中，每个值的权重由查询与对应键的兼容性函数计算得出。

3.2.1 缩放点积注意力

我们特别关注的是“缩放点积注意力机制” （图 2）。输入包括维度为 d 的查询和键，以及维度为 d 的值。我们计算查询与所有键的点积，然后将每个结果除以 $\sqrt{d}$，并应用 softmax 函数以获得权重分配于值上。

实际应用中，我们同时计算一组查询的注意力函数，这些查询被打包成矩阵 Q。键和值同样被打包成矩阵 K 和 V。输出的矩阵计算如下：

$$\text{注意力(Q, K, V)} = \text{softmax}(\frac{QK}{\sqrt{d}})V \tag{1}$$

最常用的两种注意力函数是加性注意力[2]和点积（乘法）注意力。点积注意力与我们的算法相同，除了缩放因子。加性注意力使用具有单一隐藏层的前馈网络计算兼容性函数。尽管两者在理论复杂度上相似，但在实践中，点积注意力更快且空间效率更高，因为它可以利用高度优化的矩阵乘法代码实现。

对于较小的 d 值，两种机制表现相似，但对于较大的 d 值[3]，不加缩放的加性注意力优于点积注意力。我们推测，对于较大的 d 值，点积的幅度会变得很大，将 softmax 函数推至梯度极小的区域。为了抵消这种效应，我们将点积按$\frac{1}{\sqrt{}}$进行缩放。

3.2.2 多头注意力机制

我们发现，相较于对 d 维度的键、值和查询执行单一的注意力函数，采用不同的学习线性投影分别将查询、键和值线性投影 h 次至 d、d 和 d 维度更为有益。随后，我们对这些投影后的查询、键和值并行执行注意力函数，最终得到 d 维度的输出。

为了说明点积为何会变得很大，假设 q 和 k 的分量是均值为 0、方差为 1 的独立随机变量。那么它们的点积 $q \cdot k = \sum_{i=1}^{d} q_i k_i$，其均值为 0，方差为 d。

output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_\text{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_\text{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_\text{model} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_\text{model}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_\text{model}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [38, 2, 9].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_\text{model} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

3.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_model. We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [30]. In the embedding layers, we multiply those weights by $\sqrt{d_\text{model}}$.

输出值。这些值被连接并再次投影，最终生成如图 2 所示的最终值。
多头注意力机制使模型能够同时关注来自不同位置的不同表示子空间的信息。而单一注意力头通过平均化抑制了这一点。

多头注意力(Q, K, V) = 拼接(头 1, ..., 头 n)W
其中每个头=注意力(QW_i, KW_i, VW_i)

其中投影参数矩阵为 W_i ∈ R^{d_model×d_k}, W_i ∈ R^{d_model×d_k}, W_i ∈ R^{d_model×d_v}及 W_o ∈ R^{hd_v×d_model}。

本工作中，我们采用了 h=8 个并行的注意力层（或称头）。每个头的维度设为 d= d= d/h=64。由于每个头的维度降低，总体计算成本与使用全维度的单头注意力相近。

3.2.3 注意力机制在我们模型中的应用

Transformer 在三个不同场景下运用多头注意力机制：

- 在“编码器-解码器注意力”层中，查询向量来自解码器的前一层，而记忆键与值则源自编码器的输出。这使得解码器的每个位置都能关注输入序列的所有位置，模拟了诸如[38,2,9]等序列到序列模型中典型的编码器-解码器注意力机制。
- 编码器包含自注意力层。在自注意力层中，所有的键、值和查询均来自同一处，即编码器前一层的输出。编码器中的每个位置都能关注到前一层所有位置的信息。
- 类似地，解码器中的自注意力层允许每个位置关注解码器内该位置之前（包括该位置）的所有位置。为防止信息向左流动以保持自回归特性，我们在缩放点积注意力机制内部通过掩码（设为−∞）屏蔽掉 softmax 输入中所有非法连接对应的值。参见图 2。

3.3 逐位置前馈网络

除注意力子层外，编码器和解码器的每一层还包含一个全连接的前馈网络，该网络独立且一致地应用于每个位置。其结构由两个线性变换及中间插入的 ReLU 激活函数组成。

FFN(x) = max(0, xW + b)W + b (2)

虽然线性变换在不同位置上是相同的，但它们在不同层间使用不同的参数。另一种描述方式是将之视为两个核大小为 1 的卷积操作。输入与输出的维度均为 d=512，而内部层的维度为 d=2048。

3.4 嵌入与 Softmax

与其他序列转换模型类似，我们采用可学习的嵌入将输入标记和输出标记转换为维度 d 的向量。同样地，我们使用常规的可学习线性变换及 softmax 函数，将解码器输出转化为预测的下一个标记概率。在我们的模型中，两个嵌入层与 softmax 前的线性变换共享同一权重矩阵，类似于文献[30]的做法。在嵌入层中，我们将这些权重乘以 √d.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [9].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

We also experimented with using learned positional embeddings [9] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

4 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations $(x_1, ..., x_n)$ to another sequence of equal length $(z_1, ..., z_n)$, with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [12]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence

表 1：不同层类型的最大路径长度、每层复杂度和最小顺序操作数。n 为序列长度，d 为表示维度，k 为卷积核大小，r 为受限自注意力中邻域范围。

层类型	每层复杂度	序列操作	最大路径长度
自注意力机制	O(n·d)	O(1)	O(1)
循环网络	O(n·d)	O(n)	O(n)
卷积网络	O(k·n·d)	O(1)	O(log(n))
受限自注意力	O(r·n·d)	O(1)	O(n/r)

3.5 位置编码

由于我们的模型不包含循环和卷积结构，为了让模型能够利用序列的顺序信息，我们必须注入一些关于序列中标记的相对或绝对位置的信息。为此，我们在编码器和解码器堆栈的底部向输入嵌入中添加了“位置编码”。位置编码的维度 d 与嵌入相同，以便二者可以相加。位置编码有多种选择，包括可学习的和固定的[9]。

在本工作中，我们使用了不同频率的正弦和余弦函数：

PE = sin(pos/10000)

位置编码 PE = cos(pos/10000)

其中 pos 表示位置，i 代表维度。也就是说，位置编码的每个维度对应于一个正弦波。波长构成从 2π到 10000·2π的几何级数。选择此函数是因为我们假设，模型能借此轻松学习通过相对位置进行关注——对于任意固定偏移量 k，PE 可表示为 PE 的线性函数。

我们还尝试了使用可学习的位置嵌入[9]，发现两种方法产生的结果几乎相同（见表 3 行(E)）。最终选择正弦版本，因其可能使模型外推至训练时未遇到的更长序列长度。

4 自注意力机制的缘由

本节我们将自注意力层的多个方面与常用于映射一个变长符号表示序列（x, ..., x）至另一等长序列（z, ..., z）的循环层和卷积层进行对比，其中 x, z∈ R，例如典型序列转导编码器或解码器中的隐藏层。为阐明采用自注意力的动机，我们考量了三个关键需求。

其一是每层的总计算复杂度。另一是可并行化的计算量，以所需最小顺序操作次数衡量。

第三点是网络中长距离依赖之间的路径长度。学习长距离依赖是许多序列转换任务中的核心挑战。影响学习此类依赖能力的关键因素之一，是网络中前向与后向信号必须穿越的路径长度。输入与输出序列任意位置组合间的路径越短，就越容易学习长距离依赖[12]。因此，我们还比较了由不同层类型构成的网络中，任意两个输入与输出位置之间的最大路径长度。

如表 1 所示，自注意力层通过恒定数量的顺序执行操作连接所有位置，而循环层需要 O(n)次顺序操作。就计算复杂度而言，当序列长度...（注：此处原文截断，根据语境推测后续应为"较长时，自注意力层比循环层运算更快"）

length n is smaller than the representation dimensionality d, which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [38] and byte-pair [31] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(log_k(n))$ in the case of dilated convolutions [18], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k. Separable convolutions [6], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

5 Training

This section describes the training regime for our models.

5.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [38]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

5.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models,(described on the bottom line of table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

5.3 Optimizer

We used the Adam optimizer [20] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5}) \quad (3)$$

This corresponds to increasing the learning rate linearly for the first $warmup\_steps$ training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used $warmup\_steps = 4000$.

5.4 Regularization

We employ three types of regularization during training:

当序列长度 n 小于表示维度 d 时，自注意力层比循环层更快，这在机器翻译等先进模型中使用的句子表示（如词片[38]和字节对[31]表示）中最为常见。为提升涉及超长序列任务的计算性能，自注意力可限制为仅考虑输入序列中围绕各自输出位置、大小为 r 的邻域范围。这将使最大路径长度增至 O(n/r)。我们计划在后续工作中进一步探究此方法。

当卷积核宽度 k < n 时，单层卷积无法连接所有输入与输出位置。对于连续卷积核，这需要堆叠 O(n/k)层卷积；而对于扩张卷积[18]，则需 O(log(n))层，从而增加了网络中任意两点间最长路径的长度。通常，卷积层比循环层计算成本更高，相差 k 倍。然而，可分离卷积[6]显著降低了复杂度至 O(k·n·d + n·d)。即便如此，当 k=n 时，可分离卷积的复杂度仍等同于自注意力层与逐点前馈层的组合，这正是我们模型所采用的策略。

作为额外优势，自注意力机制可产生更具解释性的模型。我们在附录中展示并分析了模型注意力分布的具体案例。不仅各个注意力头明显学会了执行不同任务，许多还表现出与句法语义结构相关的行为特征。

5 训练

本节阐述我们模型的训练方案。

5.1 训练数据与批处理

我们基于标准的 WMT 2014 英德数据集进行训练，该数据集包含约 450 万句对。句子采用字节对编码[3]处理，共享的源-目标词汇表约含 37000 个词元。对于英法翻译任务，我们使用了规模更大的 WMT 2014 英法数据集，包含 3600 万句对，并将词汇拆分为 32000 个子词单元[38]。句对按近似序列长度批量组合，每个训练批次包含约 25000 个源词元和 25000 个目标词元的句对集合。

5.2 硬件配置与训练周期

我们在配备 8 块 NVIDIA P100 GPU 的单台机器上训练模型。基础模型采用全文所述的超参数配置，每个训练步骤耗时约 0.4 秒，总计训练 100,000 步（12 小时）。大型模型（见表 3 末行配置）单步耗时 1.0 秒，共训练 300,000 步（3.5 天）。

5.3 优化器

我们采用了 Adam 优化器[20]，其参数设置为β=0.9、β=0.98 及ϵ=10。学习率在训练过程中依据以下公式动态调整：

$$\text{学习率} = d_{\text{模型}} \cdot \min(\text{步数}, \text{步数} \cdot \text{预热步数}) \quad (3)$$

这意味着在前 warmup_steps 训练步数中线性增加学习率，之后则按步数平方根的倒数比例递减。我们设定 warmup_steps=4000。

5.4 正则化

在训练过程中，我们采用了三种正则化方法：

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Residual Dropout We apply dropout [33] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.

Label Smoothing During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ [36]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

6 Results

6.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than $1/4$ the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{drop} = 0.1$, instead of 0.3.

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [38]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [38].

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU [5].

6.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the

[5] We used values of 2.8, 3.7, 6.0 and 9.5 TFLOPS for K80, K40, M40 and P100, respectively.

表 2：Transformer 在英德和英法 newstest2014 测试中以更低的训练成本取得了比以往最先进模型更优的 BLEU 分数。

模型	BLEU		训练成本（浮点运算次数）	
	英-德	英-法	英-德	英-法
ByteNet [18]	23.75			
深度注意力+位置未知 [39]	39.2		1.0·10	
GNMT+强化学习 [38]	24.6	39.92	2.3·10	1.4·10
卷积序列到序列 [9]	25.16	40.46	9.6·10	1.5·10
混合专家 [32]	26.03	40.56	2.0·10	1.2·10
深度注意力+位置未知集成 [39]	40.4		8.0·10	
GNMT+强化学习集成 [38]	26.30	41.16	1.8·10	1.1·10
卷积序列到序列集成 [9]	26.36	41.29	7.7·10	1.2·10
Transformer（基础模型）	27.3	38.1	3.3·10	
Transformer（大型模型）	28.4	41.8	2.3·10	

残差丢弃 我们在每个子层的输出上应用丢弃法[33]，然后将其与子层输入相加并进行归一化。此外，我们对编码器和解码器堆栈中嵌入向量与位置编码的和也应用了丢弃法。对于基础模型，我们使用的丢弃率为 P=0.1。

在训练过程中，我们采用了值为ϵ=0.1 的标签平滑技术[36]。这虽然会因模型学会更加不确定而增加困惑度，但能提升准确率和 BLEU 分数。

6 实验结果

6.1 机器翻译

在 WMT 2014 英语到德语翻译任务中，大型 Transformer 模型（表 2 中的 Transformer(big)）以超过 2.0 BLEU 的优势超越了之前报道的最佳模型（包括集成模型），创下了 28.4 的新 BLEU 最高分。该模型的配置详见表 3 最后一行。训练在 8 块 P100 GPU 上耗时 3.5 天完成。即便是我们的基础模型，也以远低于任何竞争模型训练成本的优势，超越了所有先前发布的模型和集成模型。

在 WMT 2014 英语到法语翻译任务中，我们的大型模型取得了 41.0 的 BLEU 分数，优于之前所有已发布的单一模型，且训练成本不到之前最先进模型的 1/4。用于英法翻译的 Transformer(big)模型采用了 P=0.1 的 dropout 率，而非 0.3。

对于基础模型，我们采用了对最后 5 个检查点取平均得到的单一模型，这些检查点每 10 分钟保存一次。对于大型模型，则平均了最后 20 个检查点。我们使用了束搜索（beam size 为 4）及长度惩罚系数α=0.6[38]。这些超参数是在开发集上经过实验选定的。在推理阶段，我们将最大输出长度设置为输入长度加 50，但允许提前终止[38]。

表 2 总结了我们的结果，并将翻译质量与训练成本同文献中的其他模型架构进行了对比。我们通过将训练时间、使用的 GPU 数量及每块 GPU 持续单精度浮点运算能力的估算值相乘，来估计训练模型所需的浮点运算次数。

6.2 模型变体

为评估 Transformer 各组件的重要性，我们以不同方式调整基础模型，测量其在英德翻译任务上性能的变化。

[5] 我们分别为 K80、K40、M40 和 P100 使用了 2.8、3.7、6.0 和 9.5 TFLOPS 的数值。

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [9], and observe nearly identical results to the base model.

6.3 English Constituency Parsing

To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes [37].

We trained a 4-layer transformer with $d_{model} = 1024$ on the Wall Street Journal (WSJ) portion of the Penn Treebank [25], about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkleyParser corpora from with approximately 17M sentences [37]. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting.

We performed only a small number of experiments to select the dropout, both attention and residual (section 5.4), learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we

表 3：Transformer 架构的变体。未列出的值与基础模型相同。所有指标均基于英语到德语的翻译开发集 newstest2013。列出的困惑度按我们的字节对编码以每词片为单位计算，不应与每词困惑度相比较。

训练 PPL BLEU 参数量 步数（开发集）	N dqh dkPdv											
(开发集) ×10 基础 6 512												
2048 8 64 64 0.1 0.1 100K 4.92												
(A)					1	512 512 5.29 24.9						
					4	128 128 5.00 25.5						
					16	32 32 4.91 25.8						
					32	16 16					5.01 25.4	
(B)						16 5.16 25.1 58						
						32					5.01 25.4 60	
(C)	2										6.11 23.7 36	
	4										5.19 25.3 50	
	8										4.88 25.5 80	
		256				32 32					5.75 24.5 28	
		1024				128 128					4.66 26.0 168	
			1024								5.12 25.4 53	
			4096								4.75 26.2 90	
(D)							0.0				5.77 24.6	
							0.2				4.95 25.5	
								0.0			4.67 25.3	
								0.2			5.47 25.7	
(E) 使用位置嵌入替代正弦函数 4.92 25.7 大型 6 1024 4096 16												
							0.3	300K	4.33	26.4	213	

开发集，newstest2013。我们采用了上一节所述的束搜索方法，但未进行检查点平均。这些结果展示在表 3 中。

在表 3 的(A)行中，我们调整了注意力头的数量及注意力键与值的维度，同时保持计算量不变，如第 3.2.2 节所述。单头注意力比最佳设置低 0.9 BLEU 分，而头数过多时质量也会下降。

在表 3 的(B)行中，我们发现减小注意力键尺寸 d 会损害模型质量。这表明确定兼容性并非易事，且比点积更复杂的兼容性函数可能有益。在(C)和(D)行中进一步观察到，正如预期，更大的模型表现更优，而 dropout 对防止过拟合极为有效。(E)行中，我们将正弦位置编码替换为学习得到的位置嵌入[9]，结果与基础模型几乎一致。

6.3 英语成分句法分析

为评估 Transformer 模型能否泛化至其他任务，我们在英语成分句法分析上进行了实验。该任务面临特定挑战：输出需遵循严格的句法结构约束，且长度显著超过输入。此外，在小数据量场景下，RNN 序列到序列模型尚未能取得最先进的性能表现[37]。

我们在宾州树库的《华尔街日报》(WSJ)语料（约 4 万条训练句子）上训练了一个 4 层 Transformer 模型（d=1024）。同时采用半监督设置进行训练，使用包含约 1700 万句子的高置信度 BerkleyParser 语料库[37]。纯 WSJ 设置采用 16K 词表，半监督设置则采用 32K 词表。

我们仅进行了少量实验以确定第 22 节开发集上的丢弃率（包括注意力机制与残差连接，见 5.4 节）、学习率和束搜索大小，其余参数均保持与英德基础翻译模型一致。在推理过程中，我们

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser el al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser el al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

increased the maximum output length to input length + 300. We used a beam size of 21 and $\alpha = 0.3$ for both WSJ only and the semi-supervised setting.

Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar [8].

In contrast to RNN sequence-to-sequence models [37], the Transformer outperforms the Berkeley-Parser [29] even when training only on the WSJ training set of 40K sentences.

7 Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

The code we used to train and evaluate our models is available at `https://github.com/tensorflow/tensor2tensor`.

Acknowledgements We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

表 4：Transformer 在英语成分句法分析中表现优异（结果基于 WSJ 第 23 节）

解析器	训练集	WSJ 23 F1 值
Vinyals & Kaiser 等人(2014)[37]	仅 WSJ，判别式模型	88.3
Petrov 等人(2006)[29]	仅 WSJ，判别式模型	90.4；Zhu 等人(2013)[40] 仅 WSJ，判别式模型 90.4；Dyer 等人(2016)[8] 仅 WSJ，判别式模型 91.7；Transformer（4 层）仅 WSJ，判别式模型 91.3；Zhu 等人(2013)[40] 半监督学习 91.3；Huang & Harper(2009)[14] 半监督学习 91.3；McClosky 等人(2006)[26] 半监督学习 92.1；Vinyals & Kaiser 等人(2014)[37] 半监督学习 92.1
Transformer（4 层）	半监督	92.7
Luong 等人(2015)[23]	多任务	93.0
Dyer 等人(2016)[8]	生成式	93.3

将最大输出长度增加至输入长度加 300。在仅使用 WSJ 数据和半监督设置下，我们均采用束宽 21 及 α=0.3 的配置。

表 4 结果显示，尽管未进行任务特定调优，我们的模型表现意外优异，除递归神经网络语法[8]外，其效果优于所有既往报道模型。
与 RNN 序列到序列模型[37]不同，Transformer 即使在仅训练 WSJ 语料库 4 万句的情况下，性能仍超越 BerkeleyParser[29]。

7 结论

在本研究中，我们提出了 Transformer 模型，这是首个完全基于注意力机制的序列转换模型，它以多头自注意力机制取代了编码器-解码器架构中常用的循环层。
对于翻译任务，Transformer 的训练速度显著快于基于循环或卷积层的架构。在 WMT 2014 英德和英法翻译任务上，我们均取得了新的技术突破。在英德翻译任务中，我们的最佳模型甚至超越了之前所有报道的集成模型性能。

我们对基于注意力机制模型的未来充满期待，并计划将其应用于更多任务。我们计划将 Transformer 扩展至处理文本以外的输入输出模态问题，并研究局部受限的注意力机制，以高效处理如图像、音频和视频等大规模输入输出。减少生成过程的序列依赖性也是我们的另一研究目标。用于模型训练与评估的代码已发布于 https://github.com/tensorflow/tensor2tensor。

致谢 我们衷心感谢 Nal Kalchbrenner 和 Stephan Gouws 富有成效的评论、修改与启发。

参考文献

[1] Jimmy Lei Ba, Jamie Ryan Kiros, 和 Geoffrey E Hinton。层归一化。arXiv 预印本 arXiv:1607.06450, 2016。

[2] Dzmitry Bahdanau, Kyunghyun Cho, 和 Yoshua Bengio。通过联合学习对齐与翻译的神经机器翻译。CoRR, abs/1409.0473, 2014。

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, 和 Quoc V. Le。神经机器翻译架构的大规模探索。CoRR, abs/1703.03906, 2017。

[4] Jianpeng Cheng, Li Dong, 和 Mirella Lapata。用于机器阅读的长短期记忆网络。arXiv 预印本 arXiv:1601.06733, 2016。

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.

[8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.

[9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.

[10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.

[13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.

[15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.

[16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.

[17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.

[18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.

[19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.

[20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.

[22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.

[23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.

[24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, 和 Yoshua Bengio。利用 RNN 编码器-解码器学习短语表示以用于统计机器翻译。CoRR, abs/1406.1078, 2014。

[6] 弗朗索瓦·乔莱。Xception：基于深度可分离卷积的深度学习。arXiv 预印本 arXiv:1610.02357，2016 年。

[7] 郑俊英、居尔切赫·恰拉尔、赵京勋与约书亚·本吉奥。门控循环神经网络在序列建模中的实证评估。CoRR，abs/1412.3555，2014 年。

[8] 克里斯·戴尔、阿迪古纳·昆科罗、米格尔·巴列斯特罗斯与诺亚·A·史密斯。循环神经网络语法。载于《北美计算语言学协会年会论文集》，2016 年。

[9] 乔纳斯·格林、迈克尔·奥利、大卫·格兰杰、丹尼斯·亚拉茨与扬·N·多芬。卷积序列到序列学习。arXiv 预印本 arXiv:1705.03122v2，2017 年。

[10] 亚历克斯·格雷夫斯。利用循环神经网络生成序列。arXiv 预印本 arXiv:1308.0850，2013 年。

[11] 何凯明、张翔宇、任少卿、孙剑。深度残差学习在图像识别中的应用——见《IEEE 计算机视觉与模式识别会议论文集》，第 770–778 页，2016 年。

[12] 塞普·霍克赖特、约书亚·本吉奥、保罗·弗拉斯科尼、于尔根·施密德胡伯。循环网络中的梯度流：学习长期依赖的困难，2001 年。

[13] 塞普·霍克赖特与于尔根·施密德胡伯。长短期记忆网络。《神经计算》，9(8):1735–1780，1997 年。

[14] 黄忠强与玛丽·哈珀。基于潜在标注的自训练概率上下文无关文法跨语言应用。载于《2009 年自然语言处理实证方法会议论文集》，第 832–841 页。计算语言学协会，2009 年 8 月。

[15] 拉法尔·约瑟福维奇、奥里奥尔·维尼亚尔斯、迈克·舒斯特、诺姆·沙泽尔与吴永辉。探索语言模型的极限。arXiv 预印本，arXiv:1602.02410，2016 年。

[16] 卢卡什·凯泽与萨米·本吉奥。主动记忆能否替代注意力机制？收录于《神经信息处理系统进展》（NIPS），2016 年。

[17] 卢卡什·凯泽与伊利亚·苏茨克弗。神经图形处理器学习算法。发表于《国际学习表征会议》（ICLR），2016 年。

[18] 纳尔·卡尔奇布伦纳、拉斯·埃斯佩霍尔特、卡伦·西蒙尼安、亚伦·范登奥尔德、亚历克斯·格雷夫斯与科雷·卡武克库奥卢。线性时间内的神经机器翻译。arXiv 预印本 arXiv:1610.10099v2，2017 年。

[19] Yoon Kim、Carl Denton、Luong Hoang 与 Alexander M. Rush。结构化注意力网络。发表于国际学习表征会议，2017 年。

[20] Diederik Kingma 与 Jimmy Ba。Adam：一种随机优化方法。发表于 ICLR，2015 年。

[21] Oleksii Kuchaiev 与 Boris Ginsburg。LSTM 网络的因子分解技巧。arXiv 预印本 arXiv:1703.10722，2017 年。

[22] 周汉林、冯敏伟、Cicero Nogueira dos Santos、余默、项兵、周博文和 Yoshua Bengio。一种结构化的自注意力句子嵌入方法。arXiv 预印本，arXiv:1703.03130，2017 年。

[23] 梁明棠、黎国伟、Ilya Sutskever、Oriol Vinyals 和 Lukasz Kaiser。多任务序列到序列学习。arXiv 预印本，arXiv:1511.06114，2015 年。

[24] 梁明棠、范辉和 Christopher D Manning。基于注意力的神经机器翻译有效方法。arXiv 预印本，arXiv:1508.04025，2015 年。

[25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

[26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.

[27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.

[28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.

[29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.

[30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

[31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.

[35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.

[36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.

[37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.

[38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.

[40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

[25] Mitchell P Marcus, Mary Ann Marcinkiewicz, Beatrice Santorini. 构建英语大规模标注语料库：宾州树库. 计算语言学, 19(2):313–330, 1993.

[26] David McClosky、Eugene Charniak 与 Mark Johnson 合著。面向解析任务的高效自训练方法。发表于北美计算语言学协会人类语言技术会议主会议论文集，第 152-159 页。计算语言学协会，2006 年 6 月。

[27] Ankur Parikh、Oscar Täckström、Dipanjan Das 及 Jakob Uszkoreit 共同提出。一种可分解注意力模型。收录于《自然语言处理实证方法》，2016 年。

[28] Romain Paulus、Caiming Xiong 与 Richard Socher 合作完成。基于深度强化学习的抽象摘要生成模型。arXiv 预印本，arXiv:1705.04304，2017 年。

[29] Slav Petrov、Leon Barrett、Romain Thibaux 与 Dan Klein。学习精确、紧凑、且可解释的树形标注方法。载于《第 21 届国际计算语言学会议暨第 44 届 ACL 年会论文集》，第 433-440 页。ACL，2006 年 7 月。

[30] Ofir Press 与 Lior Wolf。利用输出嵌入改进语言模型。arXiv 预印本 arXiv:1608.05859，2016 年。

[31] Rico Sennrich、Barry Haddow 及 Alexandra Birch。基于子词单元的稀有词神经机器翻译。arXiv 预印本 arXiv:1508.07909，2015 年。

[32] 诺姆·沙泽尔，阿扎利亚·米尔霍塞尼，克里斯托夫·马兹亚兹，安迪·戴维斯，阔克·勒，杰弗里·辛顿，杰夫·迪恩等。《超大规模神经网络：稀疏门控的专家混合层》。arXiv 预印本 arXiv:1701.06538，2017 年。

[33] 尼蒂什·斯里瓦斯塔瓦，杰弗里·E·辛顿，亚历克斯·克里泽夫斯基，伊利亚·苏茨克弗，以及鲁斯兰·萨拉赫丁诺夫。Dropout：一种防止神经网络过拟合的简单方法。《机器学习研究期刊》，15(1):1929–1958，2014 年。

[34] 赛因巴亚尔·苏赫巴塔尔，亚瑟·斯兹拉姆，贾森·韦斯顿，和罗布·弗格斯。端到端记忆网络。收录于 C. Cortes、N. D. Lawrence、D. D. Lee、M. Sugiyama 及 R. Garnett 编辑的《神经信息处理系统进展》第 28 卷，第 2440–2448 页。Curran Associates 公司，2015 年。

[35] Ilya Sutskever、Oriol Vinyals 和 Quoc VV Le。基于神经网络的序列到序列学习。载于《神经信息处理系统进展》，第 3104–3112 页，2014 年。

[36] Christian Szegedy、Vincent Vanhoucke、Sergey Ioffe、Jonathon Shlens 及 Zbigniew Wojna。重新思考计算机视觉的初始架构。《CoRR》，论文编号 abs/1512.00567，2015 年。

[37] Vinyals、Kaiser、Koo、Petrov、Sutskever 与 Hinton。将语法视为外语。《神经信息处理系统进展》，2015 年。

[38] Yonghui Wu、Mike Schuster、Zhifeng Chen、Quoc V Le、Mohammad Norouzi、Wolfgang Macherey、Maxim Krikun、曹原、高勤、Klaus Macherey 等。谷歌的神经机器翻译系统：弥合人类与机器翻译之间的鸿沟。arXiv 预印本 arXiv:1609.08144，2016 年。

[39] 周杰、曹颖、王旭光、李鹏、徐伟。带有快速前馈连接的深度循环模型在神经机器翻译中的应用。《CoRR》，论文编号 abs/1606.04199，2016 年。

[40] 朱慕华、张岳、陈文亮、张民、朱靖波。快速且准确的移进-归约成分句法分析。载于第 51 届 ACL 年会会议录（第一卷：长论文），第 434–443 页。ACL，2013 年 8 月。

Attention Visualizations

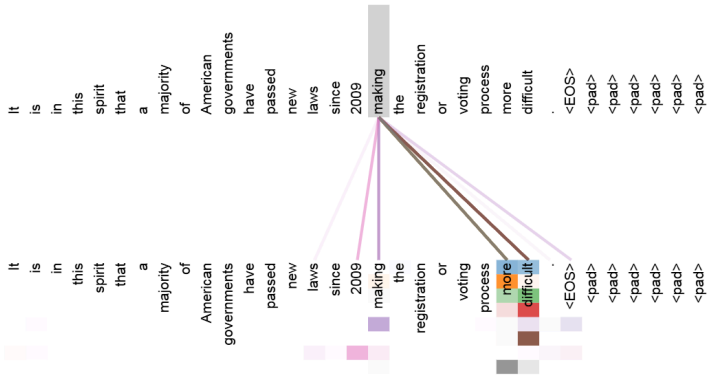


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

输入-输入层 5

注意力可视化

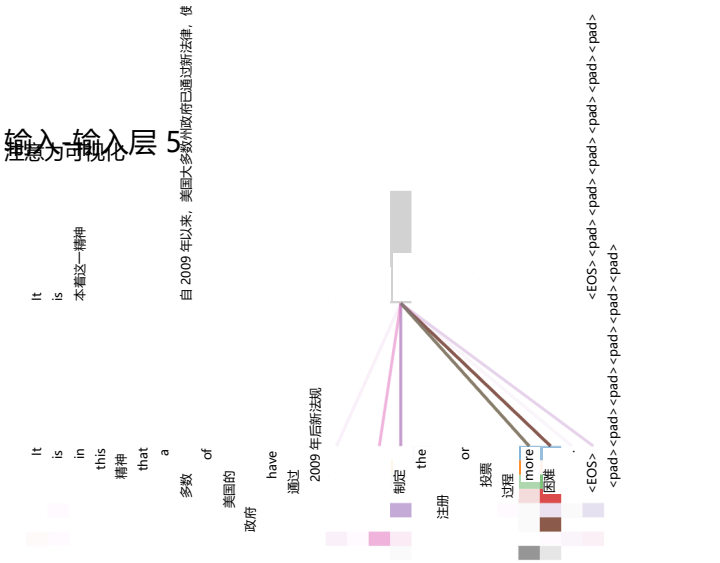


图 3：展示了第 5 层（共 6 层）编码器自注意力机制中，注意力机制遵循长距离依赖关系的一个实例。多数注意力头聚焦于动词“making”的远距离依存，补全了短语“making...more difficult”。此处仅展示单词“making”的注意力分布。不同颜色代表不同注意力头。建议彩色查看效果更佳。

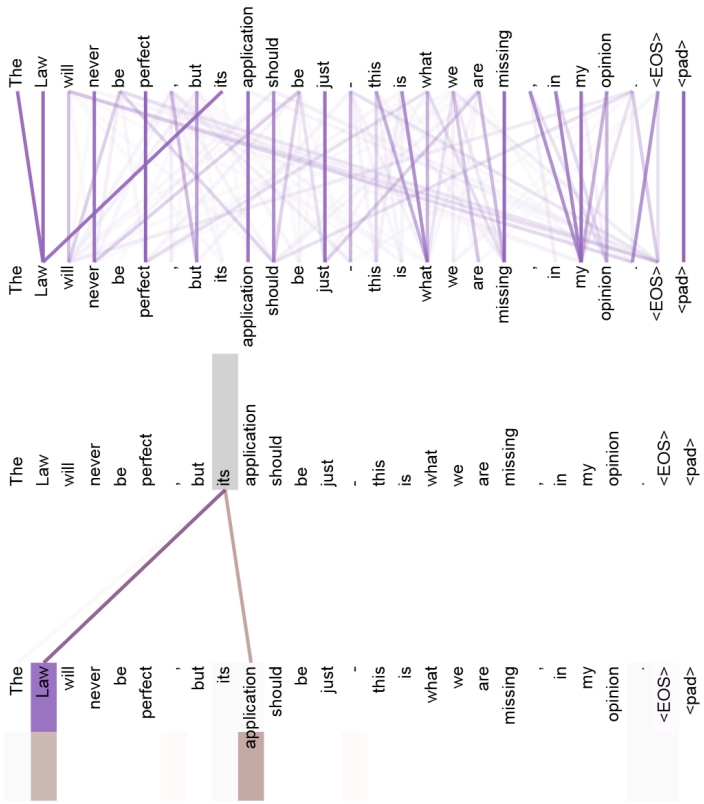


Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

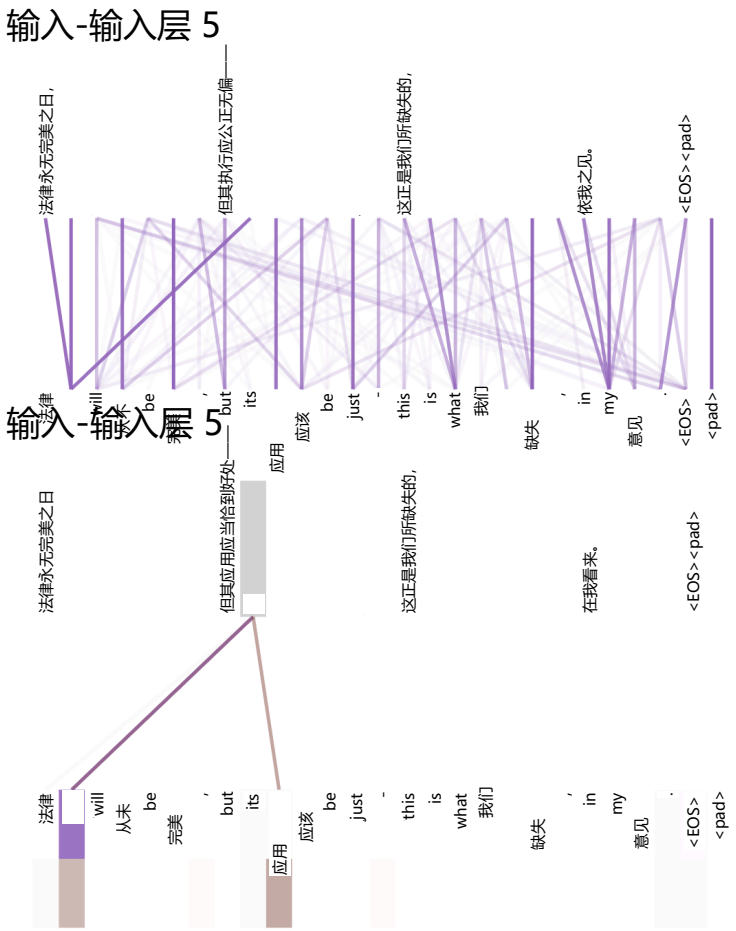


图 4：同样位于 6 层中的第 5 层，两个注意力头显然参与了指代消解任务。上图：头 5 的全部注意力分布。下图：仅针对单词 "its" 在注意力头 5 和 6 上的独立注意力分布。注意，该词的注意力分布极为集中。

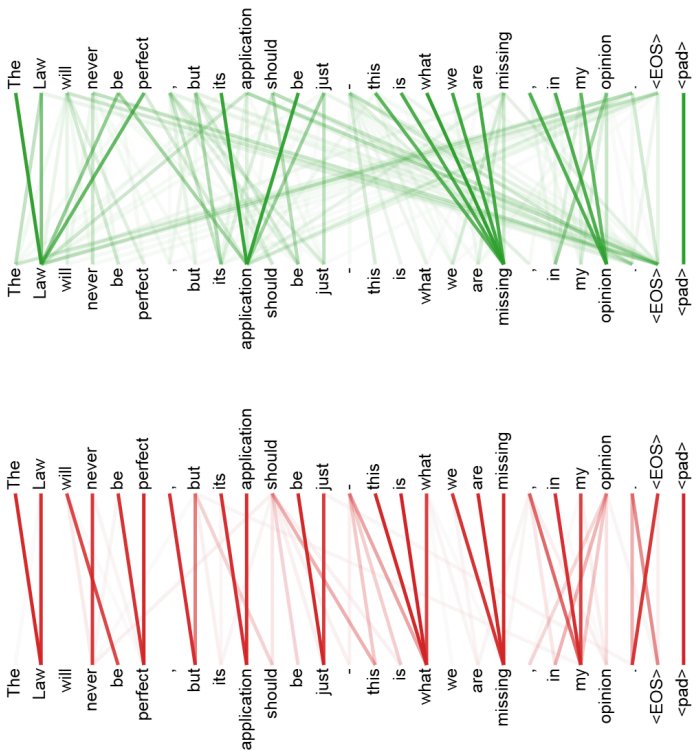


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.

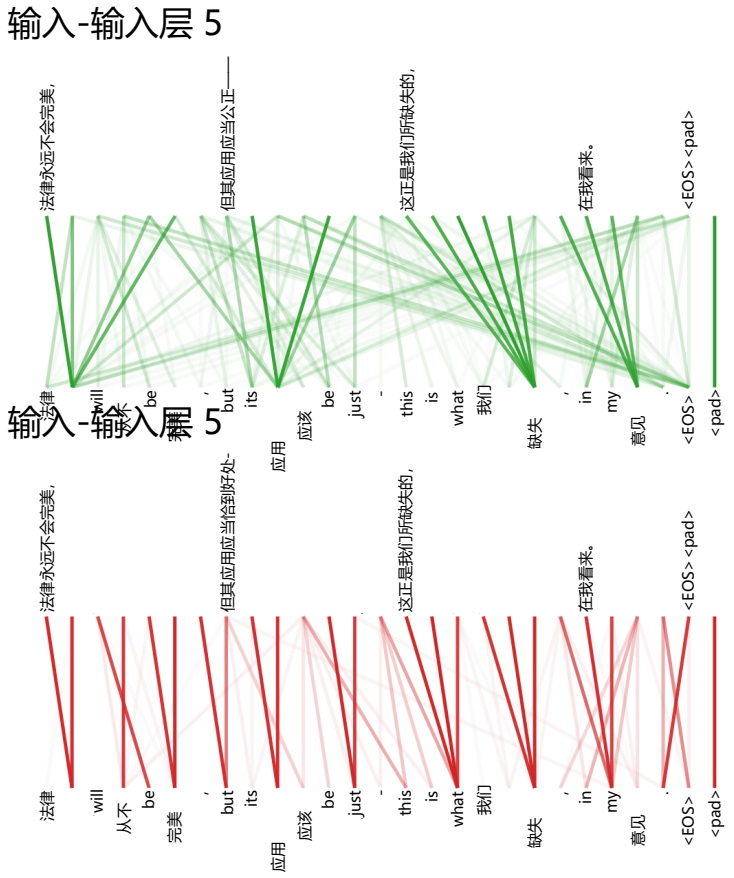


图 5：许多注意力头展现出与句子结构相关的行为模式。上文展示了两个此类示例，分别来自 6 层编码器中第 5 层的两个不同自注意力头。这些头部明显学会了执行不同的任务。